

Phase 1: The Compute Layer (Manim Rendering Node)

Overview

The compute layer is the execution engine of the zero-cost AI video generation pipeline. It is responsible for receiving dynamically generated Python scripts (Manim code) from the Large Language Model, executing them in a heavily sandboxed environment, and outputting the final MP4/GIF artifacts.

Due to the heavy computational requirements of video rendering and the severe limitations of free-tier serverless environments (specifically Hugging Face Spaces), this node is meticulously optimized for size, speed, and read-only filesystem compliance.

System Architecture & Optimizations

Building the compute layer requires adhering to six strict execution constraints to bypass traditional monolithic limitations:

1. Base Image Substitution

The official `manimcommunity/manim` Docker image is roughly 4.49 GB. Deploying this to a serverless environment like a Hugging Face Space causes fatal cold-start latency when the container wakes from a dormant state.

- **Execution:** We discard the official image and utilize `python:3.9-slim`. This provides a minimal Debian environment, allowing rapid horizontal scaling and rapid container initialization. (Alpine is avoided due to the compilation overhead of C-extensions required by graphics libraries like Cairo).

2. Dependency Pruning & Minimalist Typography

Manim relies on LaTeX for mathematical rendering. A full `texlive-full` installation can add gigabytes of unnecessary bloat.

- **Execution:** We restrict the installation strictly to the core engine (`texlive-latex-extra` , `texlive-fonts-extra` , `texlive-science` , `texlive-xetex`) and rely on the package manager to fetch specific mathematical formatting packages only on demand.

3. Container Layer Consolidation

Every `RUN` command in a Dockerfile creates a new layer, and cached package data bloats the final image size.

- **Execution:** All system installation commands (`apt-get update` , `apt-get install`) are chained together using `&&` . Crucially, the package cache is wiped (`rm -rf /var/lib/apt/lists/*`) within the exact same execution layer, preventing cache persistence.

4. Strict Filesystem Redirection (Sandboxing)

Hugging Face Spaces enforce a highly restricted, read-only filesystem to ensure security, with the sole exception being the `/tmp` directory. Furthermore, the container must run as a non-root user.

- **Execution:** The environment creates a dedicated user (`UID 1000`). Within `main.py` , every I/O operation—including the generated Python script, Manim intermediate files, FFmpeg streams, and the final MP4—is hardcoded to execute exclusively within `/tmp` .

5. Caching Suppression

Manim natively caches individual animation frames. In a constrained `/tmp` directory, this will rapidly trigger disk exhaustion errors (HTTP 500).

- **Execution:** The backend overrides this behavior by injecting the `--disable_caching` flag into the subprocess CLI command.

6. Asynchronous API Orchestration

Synchronous API connections will timeout before a heavy Manim render completes, causing edge gateways (like Cloudflare Workers) to drop the user's connection.

- **Execution:** The node runs a lightweight FastAPI web server. When the `/render` endpoint receives a task webhook from QStash, it instantly generates a Job ID, delegates the rendering subprocess to a `BackgroundTasks` thread, and immediately returns a `200 OK` response to acknowledge receipt.

Application Structure (`main.py`)

The backend is a minimalist FastAPI application designed specifically to act as a stateless webhook receiver.

Core Endpoints

- `GET /health` :
 - **Purpose:** Synthetic monitoring endpoint.
 - **Function:** Used by an external cron job (e.g., GitHub Actions) to periodically ping the server. This artificial traffic tricks the hosting provider into keeping the container

"warm," effectively preventing the instance from scaling down to zero and mitigating cold starts.

- `POST /render` :
 - **Purpose:** The primary ingestion endpoint for QStash webhooks.
 - **Payload:** Requires a JSON body containing `code` (the Manim Python script) and `scene_name` (the target class to render).
 - **Function:** Assigns a UUID, pushes the `execute_manim` function to the background, and returns the Job ID to the queue.

The Rendering Engine (`execute_manim`)

This function executes the core business logic under strict sandboxed conditions:

1. **Script Materialization:** It writes the incoming `code` string to a temporary file (`/tmp/{job_id}.py`).
2. **Subprocess Execution:** It invokes the Manim CLI via `subprocess.run()` . It explicitly enforces low quality (`-q1` for testing), routes all media to `/tmp/media_{job_id}` , and disables caching.
3. **Graceful Degradation:** If the LLM generates invalid Manim syntax, the compilation will fail. The `try-except` block catches `subprocess.CalledProcessError` and captures the exact standard error output (`e.stderr`). This trace is isolated so it can be sent back to the LLM in Phase 3 for the iterative self-correction loop.
4. **Ephemeral Cleanup:** The temporary python script is aggressively deleted in the `finally` block to conserve disk space.

Local Execution and Testing

To verify the compute layer locally before pushing to your serverless host:

1. Build the Optimized Image

```
docker build -t minimal-manim-worker .
```

2. Run the Container

```
docker run -p 7860:7860 minimal-manim-worker
```

3. Dispatch a Test Payload

```
curl -X POST http://localhost:7860/render \
-H "Content-Type: application/json" \
-d '{
    "scene_name": "TestScene",
    "code": "from manim import *\n\nclass TestScene(Scene):\n    def constru
}'
```

You should receive an immediate JSON response with a job ID, while the Docker logs will show